

# ArtConnect Pro

Database Architecture & Java Implementation

Marceau Bilger · Eliott Moriamez · Grégoire Badiche

# What is ArtConnect Pro?

## Art Community

Manage artists, artworks, galleries, exhibitions, workshops & community members in one platform.

## Relational DB

MySQL database with 13 tables, foreign keys, views, triggers, and stored procedures.

## Java Layer

JavaFX UI + Service layer + DAO interfaces backed by JDBC implementations connecting to MySQL.

# Table Relational Schema

**Artist** Core entity – name, city, disciplines, isActive

**Artwork** Linked to Artist via artistID (FK)

**ArtworkTag** Tag vocabulary (abstract, urban...)

**Tags** Artwork ↔ ArtworkTag (M:N)

**Gallery** Venue – address, hours, rating

**Exhibition** Linked to Gallery via GalleryID (FK)

**exhibits** Artwork ↔ Exhibition (M:N)

**Workshop** Title, date, capacity, level, price

**Instructor** Artist ↔ Workshop (M:N)

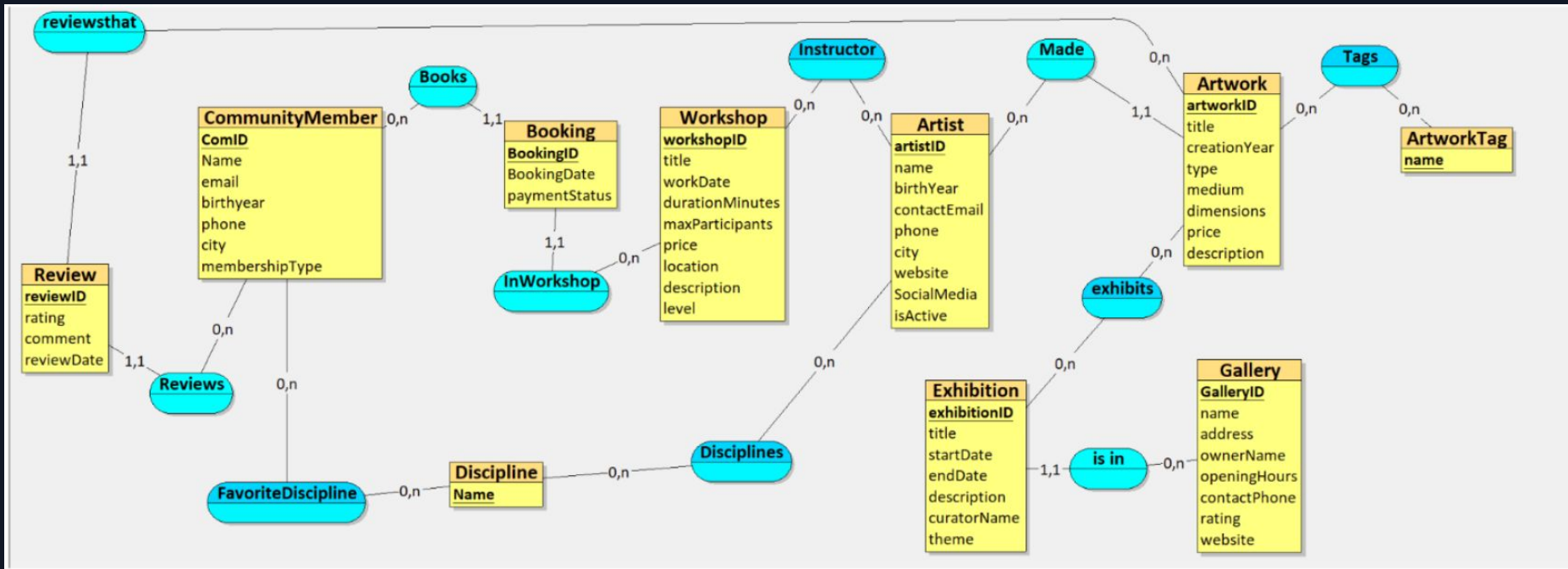
**CommunityMember** Name, city, membershipType

**Booking** Member ↔ Workshop + paymentStatus

**Review** Member reviews an Artwork (rating 1-5)

**FavoriteDiscipline** Member ↔ Discipline (M:N)

# Entity Relationships



# 3 SQL Views

## PublicArtistProfile

*Objective: Security*

Hides phone and personal email from end-users. Shows name, city, disciplines and public links only. Internal queries go directly to the Artist table (admin-restricted).

## ExhibitionCatalogue

*Objective: Simplification*

Pre-joins Exhibition → Gallery → exhibits → Artwork → Artist → Tags. Turns a 6-table query into a simple SELECT, ideal for the catalogue screen.

## ActiveExhibitions

*Objective: Security + Temporal*

Filters WHERE startDate ≤ TODAY ≤ endDate. Encapsulates time logic in one place.

# 3 Data-Integrity Triggers

## trg\_check\_workshop\_capacity

*BEFORE INSERT on Booking*

Counts existing bookings for the workshop. If `current_count ≥ maxParticipants` → raises SQLSTATE 45000 'Workshop is fully booked'.

## trg\_check\_exhibition\_dates

*BEFORE INSERT / UPDATE on Exhibition*

Ensures `endDate ≥ startDate`. Two mirrored triggers cover both INSERT and UPDATE to prevent invalid date ranges from ever reaching the table.

## trg\_check\_booking\_date

*BEFORE INSERT on Booking*

Fetches the workshop's `workDate` and compares with `CURDATE()`. Prevents members from booking workshops that have already taken place.

# Functions & Stored Procedures

## Functions

**fn\_get\_artwork\_avg\_rating(artwork)**

→ DECIMAL(4,2)

Average review rating for an artwork (0.00 if no reviews).

**fn\_get\_participant\_count(workshop)**

→ INT

Count of Paid bookings for a workshop.

**fn\_is\_member\_booked(workshop, name)**

→ BOOLEAN

Returns TRUE if member already has a booking for the workshop.

## Stored Procedures

**sp\_create\_workshop\_with\_instructor**

Validates artist is active, then wraps Workshop INSERT + Instructor INSERT in a TRANSACTION.

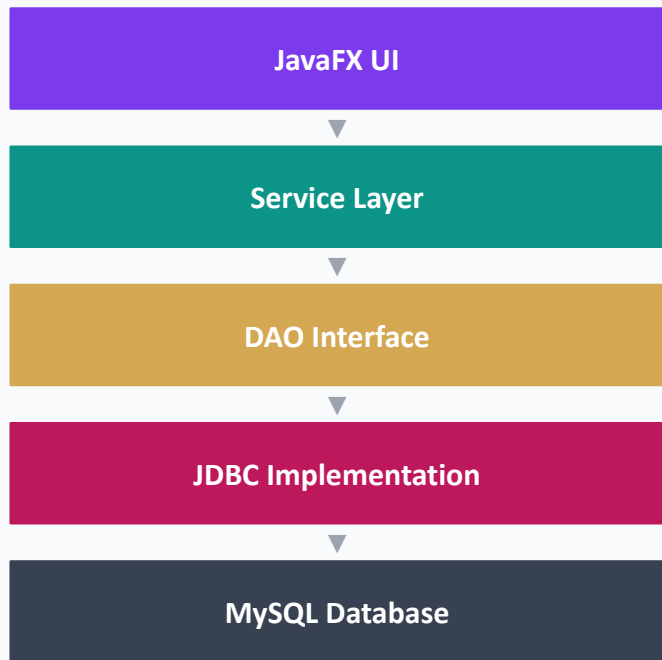
**sp\_add\_artist\_with\_discipline**

Inserts a new Artist and auto-creates the Discipline if it doesn't exist yet.

**sp\_workshop\_report**

Returns 3 result sets: workshop details, instructor list, and registered members.

# JDBC DAO Layer



## Key Design Decisions

### OOP-First, No IDs in Models

Java objects use direct references (Artwork → Artist). JDBC layer resolves DB primary keys by name at query time.

### Modular Design

The use of interfaces makes the switching of memory type (database / in memory) easy. The integration and implementation is incremental.

### API design

The whole library exposes a single API.

### Singleton Connection

ConnectionManager lazy-initialises one shared Connection; re-creates it if closed. `closeConnection()` called on shutdown.